Clase 9 · Filtro IIR en ensamblador con la FPU

Organización y Arquitectura de Computadoras · M.I. José Antonio de Jesús Arredondo Garza · lunes 5 oct 2026 · Tema 3 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. La ecuación en diferencias

Representación discreta de un sistema: la salida presente se calcula con entradas y salidas pasadas.

Yk = A0·Uk + A1·Uk−1 + A2·Uk−2 − B1·Yk−1 − B2·Yk−2

En Matlab (script `.m`): estados en cero, entrada escalón, `for i = 1:100`, calcular Yk, **después** desplazar estados (Uk−2 ← Uk−1, Uk−1 ← Uk, Yk−2 ← Yk−1, Yk−1 ← Yk) y graficar. Sirve para validar coeficientes, pero "no se asemeja en nada a la realidad" del microcontrolador.

Error común: desplazar los estados antes de calcular Yk.

2. De H(s) a H(z)

Ejemplo resuelto: **H(s) = 2/(s + 2)** con transformación bilineal y **Ts = 0.025 s → H(z) = 0.024(z + 1)/(z − 0.95)**.

Coeficiente	Valor	Nota
A0	0.024	
A1	0.024	
A2	0	Para conservar la estructura de 2.º orden
B1	**−0.95**	El signo se invierte al pasar el denominador al lado derecho
B2	0	

3. Arreglos array('f') y paso de punteros

Bloques continuos de RAM con flotantes de 32 bits (**4 bytes** cada uno):

```
U_k_S = array('f', [0.0, 0.0, 0.0])                # Uk, Uk-1, Uk-2
Y_k_S = array('f', [0.0, 0.0, 0.0])                # Yk, Yk-1, Yk-2
Constantes = array('f', [0.024, 0.024, 0.0, -0.95, 0.0])   # A0 A1 A2 B1 B2
in_ADC = array('f', [0.0])
```

Al llamar `ecuacion(U_k_S, Y_k_S, Constantes, in_ADC)`, los registros enteros reciben las **direcciones base**: **R0** ← U_k_S, **R1** ← Y_k_S, **R2** ← Constantes, **R3** ← in_ADC. El ensamblador necesita direcciones, no variables sueltas de Python.

4. La subrutina con la FPU

Decorador `@micropython.asm_thumb`, firma `def ecuacion(R0, R1, R2, R3):`.

Instrucción	Efecto
`VLDR S0, [R0, #0]`	S0 = Uk
`VLDR S1, [R0, #4]`	S1 = Uk−1
`VLDR S2, [R0, #8]`	S2 = Uk−2
`VLDR S3, [R1, #0]` … `[R1, #8]`	Yk, Yk−1, Yk−2 en S3–S5
Constantes desde R2	Desplazamientos #0, #4, #8, #12, #16
`VMUL`, `VADD`, `VSUB`	Suma de productos para la nueva Yk
`VSTR S3, [R0, #0]`	Guardar el resultado en RAM

Desplazamientos de 4 bytes

Con flotantes de 32 bits en memoria lineal, cada elemento está **4 bytes** más adelante. Usar incrementos de 1 o 2 bytes corrompe los datos.

En MicroPython corre `while True:` enviando muestras al graficador de Thonny con `print((Y_k_S,))`. Baja el cómputo a microsegundos: filtrado en tiempo real.

5. Trabajo práctico: filtro IIR de 2.º orden subamortiguado

1. Base: **$H(s) = \omega_n^2 / (s^2 + 2\zeta\omega_n s + \omega_n^2)$**.
2. **$T_s = 0.05$ s → $f_s = 20$ Hz.**
3. **Nyquist:** $f < f_s/2 = 10$ Hz ($\omega_n < 2\pi \cdot 10 \approx 62.8$ rad/s), o el filtro se vuelve inestable o con aliasing.
4. **Subamortiguado:** $\zeta < 1$ (p. ej. 0.2) para que oscile ante una onda cuadrada.
5. Sustituir los coeficientes en la plantilla `filtro_iir2` y verificar la respuesta.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Ecuación en diferencias. Yk = A0Uk + A1Uk−1 + A2Uk−2 − B1Yk−1 − B2Yk−2; calcular primero, desplazar después.

> **H(s) → H(z) (examen)**
>
> 2/(s + 2), Ts = 0.025 → 0.024(z + 1)/(z − 0.95): A0 = A1 = 0.024, A2 = 0, **B1 = −0.95** (signo invertido), B2 = 0.

Concepto	Para recordar
`array('f')`	Flotantes de 4 bytes en RAM continua
R0–R3	Direcciones base de entrada, salida, constantes, ADC
`VLDR` / `VSTR`	Cargar/guardar en S0–S31 con desplazamientos de 4
`VMUL`, `VADD`, `VSUB`	Aritmética de la FPU
Práctica	Ts = 0.05 (20 Hz), ωn < 62.8 rad/s (Nyquist), ζ < 1